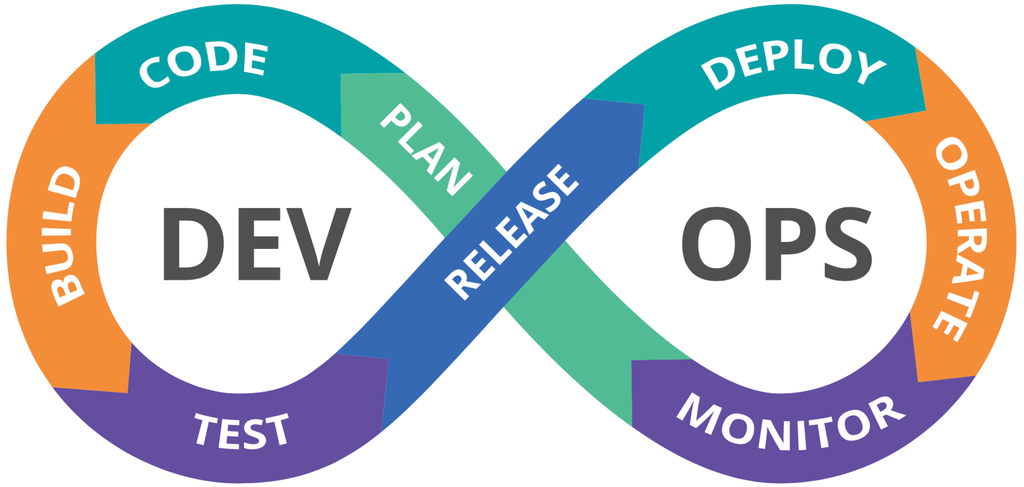
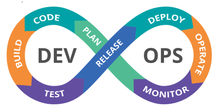


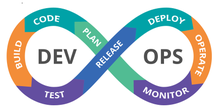
Tehničke prakse za uvođenje principa toka
Omogućavanje brzog i pouzdanog automatizovanog testiranja

predavanja su bazirana i slike preuzete iz knjige
*“The **DevOps Handbook**: How to Create World-Class Agility, Reliability, & Security in Technology Organizations”*
Gene Kim, Jez Humble, Patrick Debois, John Willis



Tehničke prakse uvođenja principe toka

- Kontinualna isporuka podrazumeva
  - uspostavljanje *deployment pipeline-a*
  - **omogućavanje brzog i pouzdanog automatizovanog testiranja**
  - omogućavanje i uvođenje u praksu kontinualne integracije
  - automatizaciju i upravljanje razvojem koji omogućava isporuku uz male rizike (*low risk deployment*)

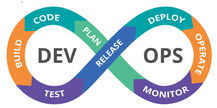


Omogućavanje brzog i pouzdanog automatizovanog testiranja

- Dosada je uspostavljena praksa korišćenja *production-like* okruženja u developmentu i QA i kod operativnih timova

- Međutim, ukoliko se testiranje izdvoji u posebnu, izdvojenu fazu i izdvojeni tim koji ima malo dodira sa developmentom što dovodi do problema:

  - Na taj način se testovi odlažu i praktično se obavljaju tek kada development “preda” svoj rad QA timu

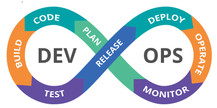


Omogućavanje brzog i pouzdanog automatizovanog testiranja

- Ukoliko se velika testiranja obavljaju nekoliko puta godišnje u velikim “paketima” testova, programeri o potencijalnim greškama saznaju prekasno, pri čemu je i sećanje o unetim promenama koje mogu izazvati uočene greške već izbledelo (što naravno otežava uklanjanje grešaka)

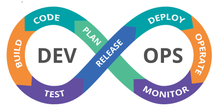
- Dodatni problem - što više koda se isporučuje, to više vremena i novca treba uložiti u testiranja - proizvodnja kompleksnih softverskih sistema postaje problematična jer postaje neskalabilna

- Automatizacija testiranja koje se obavlja nakon svakog *commit*-a je rešenje za ovakve problem



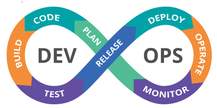
Omogućavanje brzog i pouzdanog automatizovanog testiranja

- Čak i najveći (Googlov GWS tim) su svojevremeno imali problema sa koretknim pristupom testiranju
  - veliki broj programera proizvodio je ogromne količine koda na dnevnoj bazi
  - bildovi i testiranja su trajali predugo
  - kod koji je ušao u produkciona okruženja nije bio dovoljno testiran
  - timovi su *commit*-ovali velike pakete koda koji su izazivali dosta konflikta



Omogućavanje brzog i pouzdanog automatizovanog testiranja

- Ovakvo haotično stanje je počelo da se jako odražava na timove:
  - počeli su da se boje unošenja novih izmena jer nisu razumeli celokupan sistem u potpunosti (što je i nemoguće zbog kompleksnosti)
  - iskusni programeri su takođe izbegavali da unose nove izmene - oni su, s druge strane, previše dobro znali u kakvom stanju je ceo sistem i da svaka izmena potencijalno može da donese mnogo problema
  - faktički ceo razvoj je počeo da se usporava i zazire od implementacije novih stvari



Omogućavanje brzog i pouzdanog automatizovanog testiranja

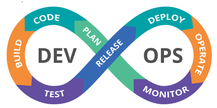
- Kako bi deblokirali ceo razvojni tim, prihvaćen je koncept uvođenja automatskih testova i to “tvrdom” filozofijom

  - Nijedna nova izmena koda neće se prihvatiti u sistem ukoliko uz nju ne idu odgovarajući automatizovani testovi, koji tu novu komponentu uspešno testiraju

  - Uspostavljen je *continious build* sistem koji je u svakom momentu morao biti u takvom stanju da svi testovi “prolaze”

  - Uspostavljen je nadzor nad pokrivenošću koda testovima (*coverage),* konstantno se insistiralo na tome da pokrivenost koda testovima stalno raste

  - Formirane su preporuke i uputstva kako kreirati testove i zahtevano je da ih se svi programeri pridržavaju (kako interni tako i eksterno angažovani)



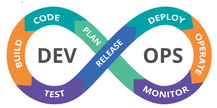
Omogućavanje brzog i pouzdanog automatizovanog testiranja

- Rezultat uvedenih promena je bilo drastično povećanje produktivnosti, uključivanje novog koda, uz ogroman broj izmena koje dolaze iz različitih timova postalo je svakodnevno

- Ovakva praksa proširena je i na ostale timove u Google-u

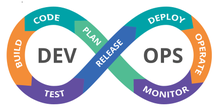
  - Kada god se komituje neki kod, to automatski pokreće mehanizam automatskog testiranja koje dati kod propušta kroz stotine ako ne i hiljade različith automatizovanih testova

  - Ukoliko svi testovi “prođu” dati kod se automatski spaja (*merge*) sa glavnom granom, postajući tako deo koda koji je spreman za puštanje u produkciju



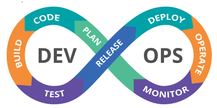
Omogućavanje brzog i pouzdanog automatizovanog testiranja

- Istovremeno ova praksa je dovela i do promene filozofije rada
  - svi su svesni da ma koliko savršen bio, pre ili kasnije će se *realease pipeline* zaustaviti zbog koda koji je pao na testovima (“u crvenom”)
  - najvažnija je saradnja da se pronađe uzrok, ispravi kod i deblokira pipeline, a ne međusobno okrivljavanje kada se zaustavljanje desi
  - svi su svesni da pri velikim količinama komitovanog koda pre ili kasnije i njihov kod može da dođe u koliziju sa nekim drugim i da će u tom slučaju i oni biti ti koji su zaustavili *pipeline*



Omogućavanje brzog i pouzdanog automatizovanog testiranja

- Ovakva praksa dovodi do neverovatnih rezultata:
  - 40000 komitova dnevno
  - 50000 bildovanja dnevno
  - 120000 automatizovanih test paketa
  - 75 miliona test slučajeva se izvršava u svakom danu
  - preko 100 inženjera radi na poslovima test inženjera, kontinualne integracije i automatizacije bild procesa



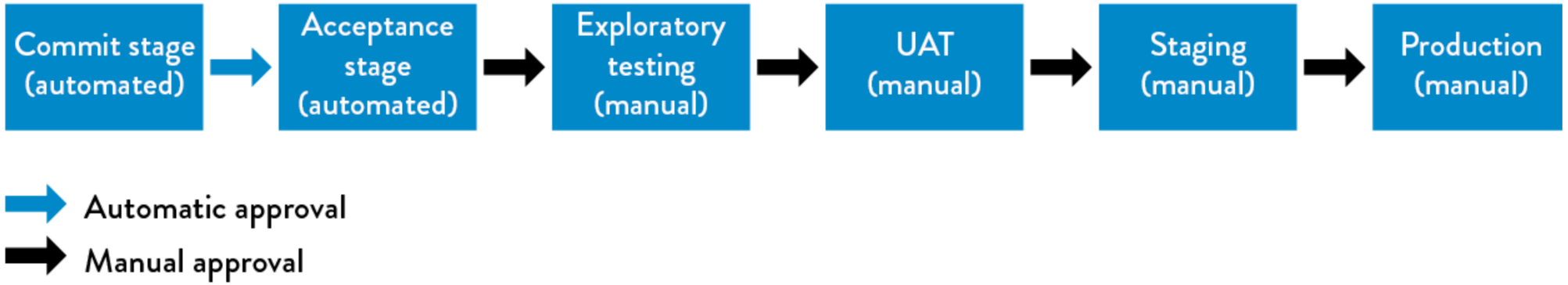
Kontinuirana izgradnja (*buildovanje*), testiranje i integrisanje koda i okruženja

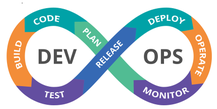
- Cilj je da se o kvalitetu proizvoda vodi računa od samog početka
- Svaki programer kreira automatizovane testove kao deo svog posla
- Na ovaj način obezbeđuje se brza povratna veza - ako nešto ne radi, to se vidi skoro odmah po komitovanju

Kontinuirana izgradnja (*buildovanje*), testiranje i integrisanje koda i okruženja

- Automatizovani test paketi (*test suites*) povećavaju frekvenciju integracije i testiranja koda, donoseći suštinsku promenu iz **povremenog** u **kontinuirano.**

- Ovo se postiže tako što se razvija *deployment pipeline* čija primaran uloga je da pokreće integraciju i automatsko testiranje čim se nova promena koda unese u sistem (komituje na sistem za kontrolu verzija)



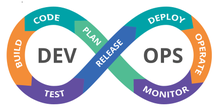


Kontinuirana izgradnja (*buildovanje*), testiranje i integrisanje koda i okruženja

- Automatizovani test paketi (*test suites*) povećavaju frekvenciju integracije i testiranja koda, donoseći suštinsku promenu iz **povremenog** u **kontinuirano.**

- Ovo se postiže tako što se razvija *deployment pipeline* čija primaran uloga je da pokreće integraciju i automatsko testiranje čim se nova promena koda unese u sistem (komituje na sistem za kontrolu verzija)

  - bildovanje i testiranje se automatski obavlja na okruženjima koja odgovaraju produkcionim



Kontinuirana izgradnja (*buildovanje*), testiranje i integrisanje koda i okruženja

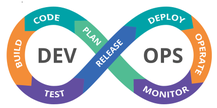
- Ovakvim pristupom svaka greška se otkriva odmah nakon što je unešena u sistem

- Da bi se ovo postiglo automatizovani procesi bildovanja i testiranja moraju da se pokreću na samo za to namenjenim (*dedicated*) okuženjima. Za to postoji više razloga:

  - proces bildovanja i testiranja može se “vrteti” stalno, bez potrebe da se prekidaju drugi poslovi

  - ovo obezbeđuje da su svi preduslovi i zavisnosti korektno podešene na ovim okruženjima i u skladu sa očekivanim stanjem na produkciji (otklanja probleme tipa “kod mene je radilo”)

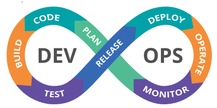
  - sve što je potrebno za izvršavanje programa može se zapakovati i po potrebi ponovno instalirati na ponovljiv način, čime se testna okruženja drže vrlosličnim stvarnim produkcionim, a kod koji prolazi na tim okruženjima morao bi da radi i u produkciji bez neugodnih iznenađenja



Kontinuirana izgradnja (*buildovanje*), testiranje i integrisanje koda i okruženja

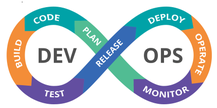
- Ovakav *pipeline* postaje i platforma na kojoj testeri obavljaju *acceptance* i *usability* testove

- Na njoj se obavljaju i testovi performansi i bezbednosni testovi (zato je bitno da je *production-like).*



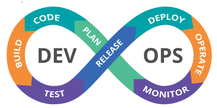
Deployment pipeline

- Postoji veliki broj alata koji obezbeđuju funkcionalnosti *deployment pipeline-a (*dosta njih je i open source*)*
  - Jenkins, ThoughtWorks Go, Concourse, Bamboo, Microsoft Team Foundation Server, TeamCity, Gitlab CI, kao i rešenja za *cloud* Travis CI Snap



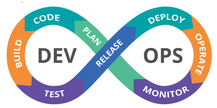
Deployment pipeline

- Proces rada pipeline-a:
  - pokreće se kada se izvrši *commit*
  - nakon čega se izvršava automatsko bildovanje i integrisanje komponenti
  - pokreću se automatizovani Unit testovi
    - mogući su i dodatni koraci provere (statička kod analiza, provera dupliranog koda, provera pokrivenosti koda testovima)
  - nakon svih uspešnih testova pokreće faza prihvatanja (*acceptance*)
    - automatski se deployuju svi prethodno kreirani paketi na *production like* okruženje i pokreću skupovi automatizovanih acceptance testova
  - nakon što prođu acceptance testovi, kod se prihvata na sistemu za verzioniranje i pokreće se pakovanje distributivne verzije (čime se garantuje da se ista distibucija koristi svuda, i uklanja problem različitih varijanti iste verzije softvera)



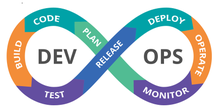
Deployment pipeline

- Deployment pipeline postaje deo neophodne infrastrukture baš kao i sistemi za verzioniranje softvera
- Čuva i kompletnu istoriju svakog bildovanja koda, uključujući informaciju o svim testovima koji su izvršeni i njihovim rezultatima
- Ove informacije su posebno vredne za brzo lociranje i otklanjanje grešaka



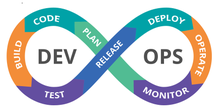
Prakse kontinuirane integracije

- Nakon što je uspostavljen *deployment pipeline,* neophodno je usvojiti praksu kontinuirane integracije za šta postoje tri pretpostavke:
  - postojanje sveobuhvatnog skupa automatizovanih testova čijim uspešnim izvršavanjem se garantuje da je kod u isporučivom (*deployable*) stanju
  - kultura rada koja podrazumeva da se obustavi rad kada se uoči da su validacioni testovi “pali”
  - programeri moraju razvijati kod u malim “paketima” koji se onda spajaju sa glavnom granom dosta često, a ne da razvijaju kod u dugotrajnim granam u kojima se razviaju komplikovane promene



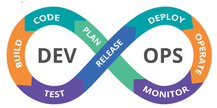
Razvoj brzih i pouzdanih validacionih testova

- Integracija i testiranje moraju se obavljati kontinualno, jer čak i periodičnost na nivou *nightly build*-a može uneti preterano kašnjenje u otkrivanju grešaka i otežati njihovo uklanjanje
- Da bi ovaj proces pouzdano radio neophodan nam je skup brzih i pouzdanih automatizovanih testova



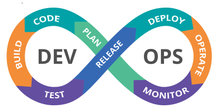
Automatizovani testovi

- **Unit testovi** (jedinični testovi)**-** tipično testiraju metodu, klasu ili funkciju u izolaciji izbegavajući eksterne zavisnosti
- **Acceptance testovi** (testovi prihvatljivosti rešenja) - Tipično testiraju aplikaciju kao celinu, potvrđujući da funkcionalnosti višeg nivoa (dakle ne samo metode/funkcije) rade na zadovoljavajući način (ovde se gledaju kriterijumi prihvatljivosti definisani u *user story*-jima, ili npr. usklađenost API-ja sa zahtevima)
  - Unit test dokazuje da deo koda radi onako kako je programer zamislio, acceptance test dokazuje da aplikacija radi onako kako je klijent tražio.
  - Svaki build koji prođe fazu automatskih acceptance testova se onda učini dostupnim za manuelno testiranje, kao i za integracione testove



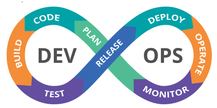
Automatizovani testovi

- **Integracioni testovi -** ovi testovi dokazuju da naša aplikacija ili modul dobro obavlja interakciju (komunikaciju) sa ostalim komponentama softverskog okruženja u kome treba da radi.
  - izvršavaju se na onim bildovima koji uspešno prođu unit i acceptance testove



Automatizovani testovi

- Zbog preopterećenosti moguće je da programeri prestanu da prave unit testove za svaki novi modul koji razvijaju
- Moraju se razviti metode za detekeciju pokrivenosti koda testovima, moguće je ugraditi pravilo da ako pokrivenost koda padne ispod određenog praga to izaziva pad testiranja
- 10-minutni bildovi sa unit testovima su prihvatljivi, unit testovi se mogu izvršavati brzo jer su jako lokalizovani

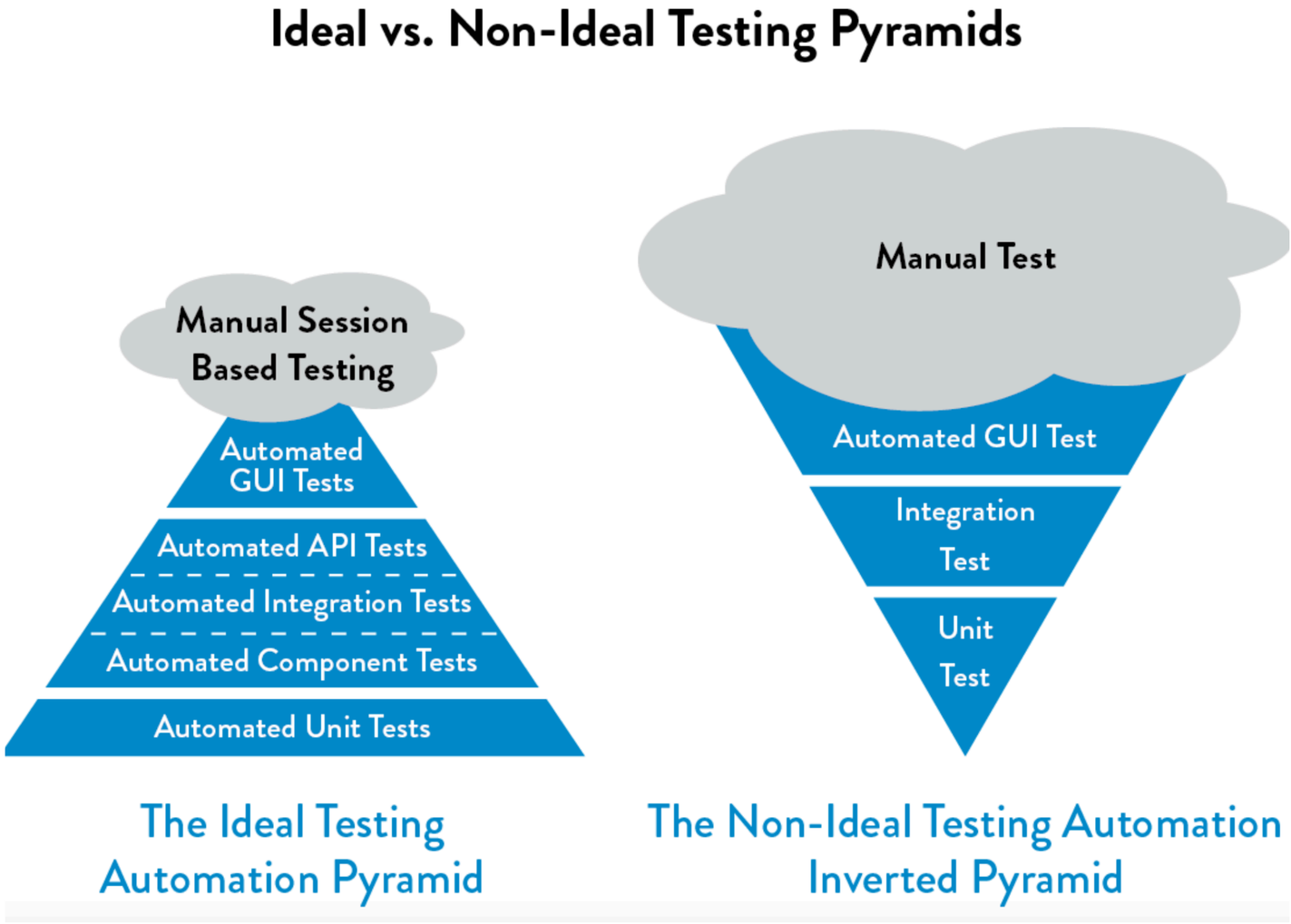


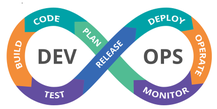
Rana detekcija grešaka automatizovanim testovima

- Smisao automatizovanih testova je da nam omoguće što raniju detekciju grešaka
- Brzi (unit testovi) rade se pre složenijih (acceptance testova), a oni opet pre bilo kakvog manuelnog testiranja
- Jedno od željenih stanja je da se svaka greška potencijalno otkrije u bržim fazama testiranja. Ako se greške otkrivaju tek u kasnijim fazama, brzina protoka povratnih informacija ka programerima jako pada, a reprodukovanje same greške je teže
- Kada god se neka greška otkrije u *acceptance* ili *integration* fazi testiranja, neophodno je razmisliti koji *unit* testovi se mogu napisati kako bi se takva greška otkrila ranije

Rana detekcija grešaka automatizovanim testovima

- *Fowler* je definisao “idealnu test piramidu” koja predstavlja takav skup testova koji omogućava da se većina grešaka pronađe u ranijim fazama



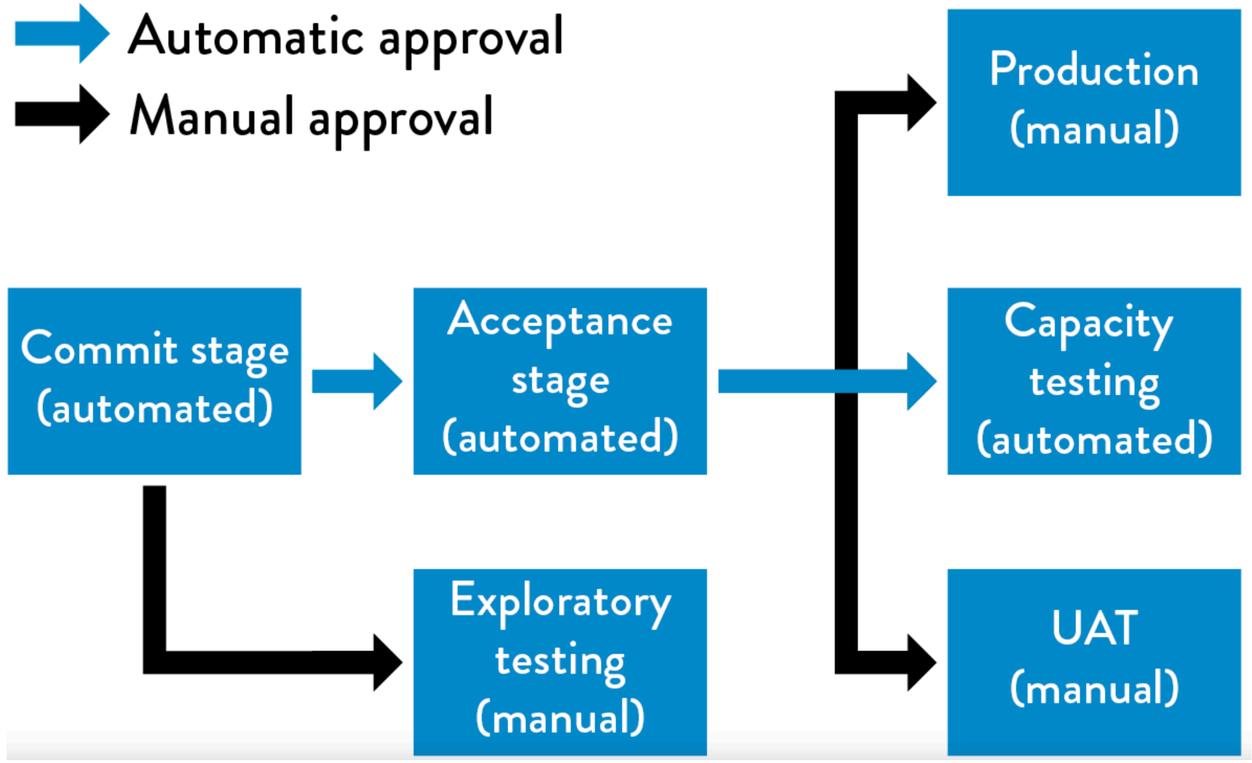


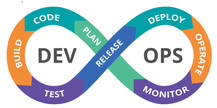
Brzo izvršavanje testova

- Pošto želimo da nam se testovi izvršavaju maksimalno moguće brzo, treba ih tako i dizajnirati i imati na umu mogućnost da se različiti testovi izvršavaju i paralelno.
- Moguće je i cele grupe testova izvršavati u paraleli - npr. kada prođemo fazu *acceptance* testova, testove performansi i sigurnosne testove možemo pokrenuti paralelno.

Brzo izvršavanje testova

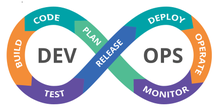
- Manuelne testove možemo dozvoliti tek što prođu svi automatski testovi čime se izbegava trošenje vremena testera na bild koji će posle “pasti” (ali ponekad želimo da izvršimo manuelne testove bez obzira na ostale)





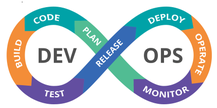
Pisanje testa pre koda (test-driven development)

- Jedan od najefikasnijih načina da se obezbedi da u svakom momentu imamo dostupne pouzdane automatizovane testove je primena ovog pristupa
  - prvo se pišu automatizovani testovi u skladu sa zahtevima i specifikacijom, a zatim se piše kod koji zadovoljava te testove
  - TDD i ATDD
  - Tri koraka
    - inicijalno stanje - napisati test koji “ne prolazi” (jer nema koda koji bi uspešno izvršio test). Komitovati kod.
    - programirati funkcionalni kod dok ne zadovolji test. Komitovati kod.
    - refakturisati i stari i novi kod kako bi bio dobro strukturisan, pazeći pri tome da testovi i dalje “prolaze”. Ponovo komitovati kod.



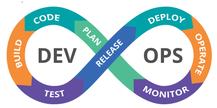
Automatizovati što više manuelnih testova

- Cilj je da se otkrivanje grešaka desi što pre - idealno u brzim fazama automatizovanih testova
- Sve koji učestvuju u testiranju - programere i testere osloboditi prostih manuelnih, ponavljajućih testiranja, a osloboditi im vreme za zahtevnije vidove manuelnih testova (npr. eksplorativno).
- Automatizacija mora napraviti testove koji su pouzdani i imaju isti ishod kao da su obavljeni manuelno - izbegavati *false positives* do kojih dolazi zbog nekih drugih razloga
- Mali skup proverenih i pouzdanih automatizovanih testova je bolja opcija od velikog skupa teških i dugotrajnih manuelnih



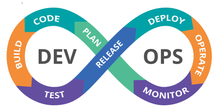
Uključiti testove performansi u test pakete

- Testovi performansi trebaju postati sastavni deo naših test paketa, čime se omogućava da se na vreme detektuju probemi sa performansama sistema u određenim okolnistima
  - Ovakvi testovi treba da provere ponašanje sistema u celosti (koda, baze podataka, sistema za smeštaj podataka (*storage*), mreže, virtuelizacije) u okolnostima opterećenja koje je slično produkcionom
  - Problem je što je ponekad okruženje za testove performansi teško napraviti



Uključiti nefunkcionalne testove u test pakete

- availability, scalability, security…
- dosta nefunkcionalnih zahteva se postiže odgovarajućim konfigurisanjem sistema, pa i za ovo treba imati odgovarajuće testove



Prekinuti razvoj kada su testovi “u crvenom”

- Analogija sa “andon cord”
- Kada je *deployment pipeline* u prekidu, rešavati problem, a obustaviti unošenje novih izmena i featura u sistem